

DivideAi

Dividir despesas em grupo de forma justa, simples e transparente

Emilly Neves & David Marques

Projeto Final da disciplina · 2025/2 · Licença MIT

`github.com/DavidMarquesss/Divideai`

`Android · Kotlin 2.0.21 · minSdk 26 · Firebase (BoM 33.1.0) · Material 3`

Propósito: que problema resolvemos?

👉 **O ponto:** não é só dividir a conta — o app **simplifica as dívidas** ao mínimo de transferências.

- **Dor:** dividir contas em grupo (viagem, república, família) vira bagunça — "quem pagou o quê" e "quem deve a quem" → **atrito**
- **Solução:** registrar cada despesa (categoria, pagador, participantes, comprovante) e **dividir automaticamente**; saldos claros: **a pagar / a receber**
- **Público-alvo:** amigos de viagem, colegas de república, famílias — **qualquer grupo que rateia custos**

Ex.: A deve 10 a B e B deve 10 a C → em vez de 2 pagamentos, o app manda **A pagar 10 direto a C**.



Ferramentas & Frameworks (visão geral)

Linguagem & Build

- Kotlin · JVM 11
- Gradle (Kotlin DSL) + Wrapper
- Version Catalog
- minSdk 26 · targetSdk 36



Android Jetpack

- ViewModel · LiveData
- Navigation · ViewBinding
- SwipeRefresh · Photo Picker
- ExifInterface · SplashScreen



UI & i18n

- Material 3
- Tema DayNight (modo escuro)
- Per-App Language (pt-BR/en)



Backend (Firebase / BaaS)

- Authentication
- Cloud Firestore (NoSQL)
- Cloud Messaging (FCM)



Bibliotecas

- MPAndroidChart (gráficos)
- ZXing (QR Code)
- notifier: Node.js + Admin SDK



Testes & DevOps

- JUnit (unitários)
- Espresso (UI)
- Git/GitHub · Issues
- CI: GitHub Actions

Modelo de domínio (o que é persistido)

👉 **Olhe:** `payerId` (quem pagou) + `participants` (quem deve quanto) — é a **entrada do algoritmo e dos testes**.

```
// data/model/Expense.kt – data class = objeto que só carrega dados
data class ExpenseShare(
    val userId: String = "",
    val amountOwed: Double = 0.0,    // quanto este usuário deve ao pagador
    val paid: Boolean = false        // já quitado?
)
data class Expense(
    val id: String = "", val groupId: String = "", val title: String = "",
    val amount: Double = 0.0, val date: String = "",
    val payerId: String = "",           // ★ quem pagou
    val participants: List<ExpenseShare> = emptyList(), // ★ entre quem dividir
    val category: String = "", val receipt: String = "" // receipt = foto em Base64
)
```

Arquitetura: camadas + MVVM

👉 **Olhe:** a tela só **observa** o estado (LiveData); a regra pesada fica fora dela → **testável**.

```
// ui/auth/LoginViewModel.kt – ViewModel guarda o estado; a tela observa
class LoginViewModel(app: Application) : AndroidViewModel(app) {
    private val repository = AuthRepository()
    private val _loginState = MutableLiveData<LoginState>()
    val loginState: LiveData<LoginState> = _loginState // ★ só-leitura p/ a UI

    fun login(email: String, password: String) = viewModelScope.launch {
        _loginState.value = LoginState.Loading
        try { repository.signIn(email, password); _loginState.value = LoginState.Success }
        catch (e: Exception) { _loginState.value = LoginState.Error(/* msg traduzida */) }
    }
}
// LoginActivity: viewModel.loginState.observe(this) { render(it) }
```

★ Algoritmo estrela: DebtSimplifier (Kotlin puro)

👉 Olhe: o loop pega sempre o **maior credor × maior devedor** (★). Por quê: é o que dá o mínimo de transferências.

```
// 1) Saldo líquido de cada usuário (+ tem a receber, - tem a pagar)
for (expense in expenses) for (share in expense.participants) {
    if (share.paid || share.userId == expense.payerId) continue
    balance.merge(expense.payerId, share.amountOwed, Double::plus) // credor +
    balance.merge(share.userId, -share.amountOwed, Double::plus) // devedor -
}
```

```
// 2) Loop guloso: maior credor × maior devedor, até zerar
while (true) {
    val creditor = balance.maxByOrNull { it.value } ?: break // ★ maior credor
    val debtor = balance.minByOrNull { it.value } ?: break // ★ maior devedor
    if (creditor.value <= EPSILON || debtor.value >= -EPSILON) break
    val amount = min(creditor.value, -debtor.value)
    transfers += SettlementTransfer(debtor.key, creditor.key, amount)
    balance[creditor.key] = creditor.value - amount
    balance[debtor.key] = debtor.value + amount // ... remove quem zerou
}
```

Build ⚙️ — Gradle + Wrapper (Kotlin DSL)

👉 **Por que importa:** o Wrapper fixa a versão do Gradle no repo (★) → `git clone` + `./gradlew` compila em qualquer máquina, sem instalar nada.

- **Build** = virar o código em **APK**; o **Gradle** baixa libs, compila e empacota (config em `.kts` , Kotlin)

```
# gradle/wrapper/gradle-wrapper.properties
distributionUrl=https\://services.gradle.org/distributions/gradle-8.11.1-bin.zip # ★
```

```
// build.gradle.kts (raiz) – plugins declarados 1x, aplicados nos módulos
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.android) apply false
    id("com.google.gms.google-services") version "4.4.4" apply false
}
```

Build ⚙️ — dependências: catálogo + repositórios

👉 **Olhe:** versões centralizadas no catálogo (★ muda 1 linha); os repositórios dizem **de onde** o Gradle baixa as libs.

```
# gradle/libs.versions.toml
[versions]
agp = "8.10.1"
kotlin = "2.0.21"    # ★ muda aqui
coreKtx = "1.17.0"
junit = "4.13.2"
[libraries]
androidx-core-ktx = { group =
    "androidx.core", name = "core-ktx",
    version.ref = "coreKtx" }
[plugins]
android-application = { id =
    "com.android.application",
    version.ref = "agp" }
```

```
// settings.gradle.kts
dependencyResolutionManagement {
    repositoriesMode.set(
        FAIL_ON_PROJECT_REPOS) // sem repo solto
    repositories {
        google()
        mavenCentral()
        maven { url =
            uri("https://jitpack.io") } // MPAndroidChart
    }
}
include(":app")
```


Build ⚙️ — dependências do módulo + Firebase BoM

👉 **Olhe:** o `platform(firebase-bom)` (★) faz `auth` e `messaging` entrarem **sem número de versão**.

```
// app/build.gradle.kts
android {
    namespace = "com.example.divideai"; compileSdk = 36
    defaultConfig { minSdk = 26; targetSdk = 36
        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner" }
    buildFeatures { viewBinding = true } // habilita o ViewBinding
}
dependencies {
    implementation(libs.androidx.core.ktx) // alias do catálogo
    implementation(libs.material) // Material 3
    implementation(platform("com.google.firebase:firebase-bom:33.1.0")) // ★ alinha versões
    implementation("com.google.firebase:firebase-auth-ktx") // sem versão (BoM)
    implementation("com.google.firebase:firebase-messaging-ktx") // sem versão
    implementation("com.github.PhilJay:MPAndroidChart:v3.1.0") // gráficos
    implementation("com.journeyapps:zxing-android-embedded:4.3.0") // QR Code
    testImplementation(libs.junit) // JUnit 4 (testes)
}
```

Frameworks — Firebase: Auth + Firestore

👉 **Olhe:** login vira `suspend` (esperar sem travar) e o banco lê com `toObject` (★), virando objeto Kotlin.

```
// AuthRepository.kt – login (serviço pronto)
suspend fun signIn(email: String, pass: String)
: Result<AuthResult> =
suspendCancellableCoroutine { cont ->
    auth.signInWithEmailAndPassword(email, pass)
        .addOnCompleteListener { t ->
            if (t.isSuccessful)
                cont.resume(Result.success(t.result!!))
            else cont.resumeWithException(t.exception!!)
        }
    }
}
```

```
// ExpenseRepository.kt – Firestore (NoSQL)
private val db = Firebase.firestore
fun addExpense(e: Expense, cb:(Boolean,String?)->Unit){
    val doc = db.collection("expenses").document()
    doc.set(e.copy(id = doc.id)) // grava
        .addOnSuccessListener { cb(true, null) }
        .addOnFailureListener { cb(false, it.message) }
}
// leitura: get() → it.toObject(Expense::class.java) // ★
```

Frameworks — Firebase: Cloud Messaging (FCM)

👉 Olhe: o **token** do aparelho é salvo em `users/{uid}.fcmToken` (★) — é o "endereço" pro servidor mandar o push.

```
// MainActivity – registra o token FCM deste aparelho no Firestore
FirebaseMessaging.getInstance().token.addOnSuccessListener { token ->
    FirebaseFirestore.getInstance()
        .collection("users").document(uid).update("fcmToken", token)    // ★
}
```

```
// notifications/DivideAiMessagingService.kt – recebe o push
class DivideAiMessagingService : FirebaseMessagingService() {
    override fun onNewToken(token: String) = saveTokenToFirestore(token)
    override fun onMessageReceived(message: RemoteMessage) {
        val title = message.notification?.title ?: message.data["title"] ?: defaultTitle
        val body = message.notification?.body ?: message.data["body"].orEmpty()
        showNotification(title, body)          // NotificationCompat.Builder(...)
    }
}
```

Frameworks 🛔 — UI: Navigation + ViewBinding + Material 3

👉 Olhe: **um** `setupWithNavController` (★) liga a barra inferior ao "mapa" de telas; ViewBinding = sem `findViewById`.

```
// MainActivity – ViewBinding + BottomNavigation ligada ao Navigation Component
binding = ActivityMainBinding.inflate(layoutInflater) // ViewBinding (acesso tipado)
setContentView(binding.root)
val navHost = supportFragmentManager
    .findFragmentById(R.id.nav_host_fragment_activity_main) as NavHostFragment
navView.setupWithNavController(navHost.navController) // ★ liga barra ↔ grafo
```

```
<!-- res/navigation/mobile_navigation.xml – o "mapa" das telas -->
<navigation app:startDestination="@+id/navigation_groups">
    <fragment android:id="@+id/navigation_groups"
        android:name="com.example.divideai.ui.groups.GroupsFragment" />
    <fragment android:id="@+id/navigation_expenses" android:name="...MyExpensesFragment" />
</navigation>
```

Frameworks 🔌 — Bibliotecas: gráficos + QR Code

👉 **Olhe:** damos os dados e a lib desenha a pizza (★); a mesma URI do convite é gerada e lida no QR.

```
// MPAndroidChart – pizza do dashboard
val entries = state.breakdown.map {
    PieEntry(it.total.toFloat(),
        getString(it.category.labelRes)) // ★ dado→fatia
}
val ds = PieDataSet(entries, "").apply {
    colors = state.breakdown.map { it.colorArgb }
    valueFormatter =
        PercentFormatter(binding.pieChart) // %
}
binding.pieChart.data = PieData(ds)
binding.pieChart.animateY(800)
```

```
// ZXing – gerar e ler o QR do convite
val bmp = BarcodeEncoder().encodeBitmap(
    GroupInviteCode.encode(groupId), // URI
    BarcodeFormat.QR_CODE, 640, 640)
// ler (Activity Result API):
registerForActivityResult(ScanContract()){ r->
    val id = GroupInviteCode.decode(r?.contents)
    ?: return@registerForActivityResult
    joinGroupFromInvite(id) // entra
}
```

Frameworks — Jetpack: imagem (Base64) + i18n

👉 **Por que importa:** a imagem vira **Base64** (texto) e cabe no Firestore (★) → dispensa o Firebase Storage.

```
// Photo Picker (sem permissão de Storage) + Base64Image (reduz, EXIF, comprime)
private val pick = registerForActivityResult(ActivityResultContracts.PickVisualMedia()) {
    uri -> if (uri != null) handlePickedReceipt(uri) }

fun encodeFromUri(context, uri, maxSize = 720, quality = 70): String? {
    val scaled = decodeScaled(context, uri, maxSize) // reduz resolução
    val rotated = applyExifOrientation(context, uri, scaled) // corrige orientação
    return Base64.encodeToString(toJpegBytes(rotated, quality), Base64.NO_WRAP) // ★
}
```

- **i18n (pt-BR/en):** seletor **dentro do app** com `AppCompatActivity.setApplicationLocales(locales)` (persiste)

Testes 🧪 — estrutura e cobertura

👉 **Cobertura: 23 testes** unitários em **4 áreas** da lógica do app, todos passando — rodam na **JVM, sem emulador** (`./gradlew test` , 🌟).

- **Teste automatizado** = código que confere o código sozinho. **JUnit** = unitário (na máquina) · **Espresso** = interface (emulador)
- Ficam em `app/src/test/...` (unitários, sem device) vs `app/src/androidTest/...` (tela). Testamos a **regra de negócio**, mantida separada da UI justamente pra ser testável:

Classe testada (<code>app/src/test/...</code>)	Parte do app	Casos
<code>DebtSimplifier</code>	acerto de contas — quem paga quem	5
<code>ExpenseSplit</code>	divisão da despesa (pagador + participantes)	6
<code>GroupInviteCode</code>	convite de grupo por QR Code	6
<code>ExpenseCategory</code>	categorias da despesa	6

Testes 🧪 — DebtSimplifierTest: os 5 cenários

👉 **Olhe:** cada `@Test` monta um caso e o `assertEquals` **afirma o resultado esperado** — se sair diferente, o build falha.

```
// debt("B","A",10.0) = "B pagou por A" → A deve 10 a B
@Test fun `cadeia A deve B e B deve C vira transferencia A para C`() {
    val t = DebtSimplifier.simplify(listOf(debt("B","A",10.0), debt("C","B",10.0)))
    assertEquals(1, t.size)           // ★ sobra 1 pagamento só
    assertEquals("A", t[0].debtorId) // A paga...
    assertEquals("C", t[0].creditorId) // ...direto pro C – o B some
}
```

Cenário (o caso que o teste monta)	O que ele garante
★ Cadeia — A deve a B e B deve a C	vira 1 só pagamento A → C (o do meio some)
Dívida direta — A pagou R\$25 por B	1 transferência: B → A, R\$25
Rateio — jantar de R\$30 entre A, B e C	B e C devem R\$10 cada a A (a parte de quem pagou é ignorada)
Parcela já paga (<code>paid = true</code>)	0 transferências — sai do cálculo
Lista de despesas vazia	0 transferências — não quebra

Opcionais — Issue tracking + CI/CD (GitHub Actions)

👉 **Por que importa:** todo push roda `./gradlew test` (★) → se o algoritmo quebrar, o CI acusa antes do merge.

- **Issue tracking:** GitHub Issues · **CI** publica **relatório de testes** e **APK** como artefatos

```
# .github/workflows/android.yml
on:
  push: { branches: [ main ] }
  pull_request: { branches: [ main ] }
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '17' }
      - uses: android-actions/setup-android@v3
      - run: ./gradlew test          # ★ roda os 23 testes (a suíte toda)
      - run: ./gradlew assembleDebug # gera o APK de debug
```

Opcionais — Containers + o extra: notifier (Node.js)

👉 Olhe: `onSnapshot` (★) reage a despesa nova em tempo real e notifica os participantes — de **fora** do app (Node).

- **Containers:** *não feito* — app Android não roda em container; o `notifier/` poderia ser containerizado (evolução)
- **Extra:** `notifier/` em **Node.js** com `firebase-admin` (Admin SDK) → projeto **poliglota** (Kotlin + JS)

```
// notifier/watch.js – escuta o Firestore em tempo real e notifica
db.collection('expenses').where('createdAt', '>=', startedAt)
  .onSnapshot((snap) => {                                     // ★ reage em tempo real
    for (const change of snap.docChanges()) {
      if (change.type !== 'added') continue;
      for (const p of (change.doc.data().participants || [])) {
        if (p.userId === change.doc.data().payerId) continue; // não avisa quem pagou
        sendToUser(p.userId, 'Nova despesa no grupo', 'Você foi incluído.');
```

Demonstração ao vivo (opcional)

👉 O clímax: mostrar o pulo de **Saldos** → **Saldos Simplificados**.

1. Login → Criar grupo ("Viagem Praia")
2. Convidar membro por **QR Code**
3. Lançar despesa: **categoria + pagador + participantes + comprovante**
4. Ver **Saldos** → **SALDOS SIMPLIFICADOS**
5. **Dashboard**: gráfico de pizza por categoria
6. (*opcional*) rodar o `notifier` → push chegando

Plano B: prints em `docs/screenshots/` (01 → 05) seguem o mesmo fluxo se o emulador falhar.

Encerramento

1. **Problema real:** dividir contas em grupo e saber "quem deve a quem" gera confusão
2. **Solução:** registrar, dividir e **SIMPLIFICAR** as dívidas ao mínimo de transferências (`DebtSimplifier` , Kotlin puro, **testado**)
3. **Stack sólida:** Android nativo + Jetpack + Material 3 + Firebase; build reproduzível (Wrapper + Version Catalog); **testes + CI**

Diferenciais: testes + CI · QR Code · Base64 no Firestore · modo escuro · i18n · notifier Node.js

Próximos passos: testes de interface (Espresso) · containerizar o notifier · marcar dívida como paga

Obrigado! 🐮

Perguntas?

Emilly Neves & David Marques

Projeto Final 2025/2 · Licença MIT

github.com/DavidMarquesss/Divideai